



High-Performance Grid-Based Environmental Spread Modeling

Using Multithreaded CPU Parallelism

LANGUAGE

C++ / OpenMP

GRID CELLS

Up to 360,000

MAX THREADS

8 Cores

SIMULATIONS

200 Steps

Comparing Sequential vs. Parallel execution across varying grid sizes and thread counts

Project Overview

Problem Domain

Natural phenomena — heat diffusion & forest fire spread — can be modeled on 2D grids where each cell evolves based on neighbors over discrete time steps.

Approach

Grid is discretized into cells (EMPTY / TREE / BURNING). Each step applies probabilistic fire-spread rules. Since cell updates are independent, parallelism is a natural fit.

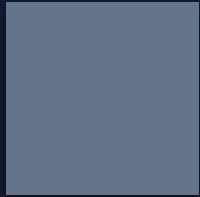
Goal

Implement both sequential and OpenMP-parallel versions; measure execution time, speedup, and scalability across grid sizes (200–600) and thread counts (1–8).

Tech Stack

C++17 · OpenMP · chrono (high-res timing) · Fixed seed (srand(42)) for reproducibility · Static scheduling with collapse(2) for 2D loops

How the Simulation Works



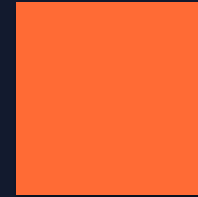
EMPTY

State = 0



TREE

State = 1



BURNING

State = 2

Update Rules (per step)

1. BURNING → EMPTY (burned out after one step)
2. TREE with a BURNING neighbor → BURNING (fire spreads to 4-connected neighbors)
3. TREE with no burning neighbor → TREE (stays green)
4. EMPTY → EMPTY (stays empty — no regrowth in this model)

💡 Fire always starts at `grid[0][cols/2]` — center of the top row

Actual Performance Results

Grid Size	Threads	Mode	Time (s)	Speedup
200×200	1	Parallel	0.4816	0.9731
200×200	2	Parallel	0.2855	1.6415
200×200	4	Parallel	0.1761	2.6609
200×200	8	Parallel	0.1863	2.5157
200×200	1	Sequential	0.4687	1.0000
400×400	1	Parallel	2.1822	0.9761
400×400	2	Parallel	1.1578	1.8397
400×400	4	Parallel	0.6971	3.0554
400×400	8	Parallel	0.6390	3.3335
400×400	1	Sequential	2.1299	1.0000
600×600	1	Parallel	5.0303	0.9748
600×600	2	Parallel	2.6078	1.8804
600×600	4	Parallel	1.5256	3.2142
600×600	8	Parallel	1.4152	3.4651
600×600	1	Sequential	4.9036	1.0000

```
(kali@kali)-[~/Desktop/Parallel Project]
$ ./test
Grid Size | Threads | Parallel | Time (s) | Speedup
200×200 | 1 | Yes | 0.4816 | 0.9731
200×200 | 2 | Yes | 0.2855 | 1.6415
200×200 | 4 | Yes | 0.1761 | 2.6609
200×200 | 8 | Yes | 0.1863 | 2.5157
200×200 | 1 | No | 0.4687 | 1.0000
400×400 | 1 | Yes | 2.1822 | 0.9761
400×400 | 2 | Yes | 1.1578 | 1.8397
400×400 | 4 | Yes | 0.6971 | 3.0554
400×400 | 8 | Yes | 0.6390 | 3.3335
400×400 | 1 | No | 2.1299 | 1.0000
600×600 | 1 | Yes | 5.0303 | 0.9748
600×600 | 2 | Yes | 2.6078 | 1.8804
600×600 | 4 | Yes | 1.5256 | 3.2142
600×600 | 8 | Yes | 1.4152 | 3.4651
600×600 | 1 | No | 4.9036 | 1.0000
(kali@kali)-[~/Desktop/Parallel Project]
$
```

* Actual results measured on test machine (srand(42), 200 steps per run)

```

[|||||] 100.0% Tasks: 83, 244 thr, 134 MIB; 4 running
[|||||] 100.0% Load average: 0.99 0.39 0.15
[|||||] 100.0% Uptime: 00:09:43
[|||||] 100.0%
Mem [|||||] 325M/4.00G
Swp [|||||] 0K/154K

```

```

Main 170
PID USER PRI NI VIRT RES SHR D CPU/MEM/IO TIME+ Command
6345 kali 20 0 58692 7132 3988 R 101.6 0.2 0:22.01 ./test
6354 kali 20 0 58696 7136 0 R 98.4 0.2 0:06.61 ./test
6528 kali 20 0 58696 7136 0 R 91.2 0.2 0:01.40 ./test
6527 kali 20 0 58696 7136 0 R 90.6 0.2 0:01.39 ./test
5906 kali 20 0 6952 5216 3612 R 2.0 0.1 0:01.30 httpd
902 root 20 0 494M 177M 85912 S 0.7 4.3 0:15.24 /usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run
1354 kali 20 0 103M 62320 21156 S 0.7 1.5 0:01.03 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.
1356 kali 20 0 103M 62320 21156 S 0.7 0.7 0:01.42 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.
5924 kali 20 0 780M 67908 50864 S 0.7 1.6 0:08.52 /usr/bin/qterminal
1 root 20 0 24008 14676 10944 S 0.0 0.3 0:01.40 /sbin/init splash
493 root 20 0 47064 15232 11780 S 0.0 0.4 0:00.43 /usr/lib/systemd/systemd-journald
419 root 20 0 155M 1920 1604 S 0.0 0.8 0:00.00 vmware-vblock-fuse /run/vblock-fuse -o rw,subt
420 root 20 0 155M 1920 0 S 0.0 0.0 0:00.00 vmware-vblock-fuse /run/vblock-fuse -o rw,subt
421 root 20 0 155M 1920 0 S 0.0 0.0 0:00.00 vmware-vblock-fuse /run/vblock-fuse -o rw,subt
422 systemd-t 20 0 90404 7628 8492 S 0.0 0.2 0:00.05 /usr/lib/systemd/systemd-timesyncd
427 root 20 0 90404 7628 8264 S 0.0 0.3 0:00.47 /usr/lib/systemd/systemd-udev
428 systemd-t 20 0 90404 7628 0 S 0.0 0.2 0:00.00 /usr/lib/systemd/systemd-timesyncd
564 root 20 0 8592 4940 1780 S 0.0 0.1 0:00.14 /usr/sbin/haveged --foreground --verbose=1
571 root 20 0 111M 5884 8316 S 0.0 0.2 0:00.65 /usr/bin/vmtoolsd
593 root 20 0 101M 7328 6736 S 0.0 0.2 0:00.09 /usr/libexec/accounts-daemon
601 messagebus 20 0 10176 7160 1116 S 0.0 0.2 0:00.30 /usr/bin/dbus-daemon --system --address=systemd:
606 polkitd 20 0 173M 9988 7400 S 0.0 0.2 0:00.17 /usr/lib/polkit-1/polkitd --no-debug --log-level
625 root 20 0 7080 3832 1588 S 0.0 0.1 0:00.01 /usr/sbin/cron -f
633 root 20 0 18660 8200 7824 S 0.0 0.2 0:00.16 /usr/lib/systemd/systemd-logind

```

```

(kali@kali)~(/Desktop/Parallel Project)

```

```

$ ./test
Grid Size | Threads | Parallel | Time (s) | Speedup
200x200 | 1 | Yes | 0.4816 | 0.9731
200x200 | 2 | Yes | 0.2855 | 1.6415
200x200 | 4 | Yes | 0.1761 | 2.6609
200x200 | 8 | Yes | 0.1863 | 2.5157
200x200 | 1 | No | 0.4687 | 1.0000
400x400 | 1 | Yes | 2.1822 | 0.9761
400x400 | 2 | Yes | 1.1578 | 1.8397
400x400 | 4 | Yes | 0.6971 | 3.0554
400x400 | 8 | Yes | 0.6390 | 3.1335
400x400 | 1 | No | 2.1299 | 1.0000
600x600 | 1 | Yes | 5.0303 | 0.9748
600x600 | 2 | Yes | 2.6078 | 1.8894
600x600 | 4 | Yes | 1.5256 | 3.2142
600x600 | 8 | Yes | 1.4152 | 3.4651
600x600 | 1 | No | 4.9836 | 1.0000

```

```

(kali@kali)~(/Desktop/Parallel Project)

```

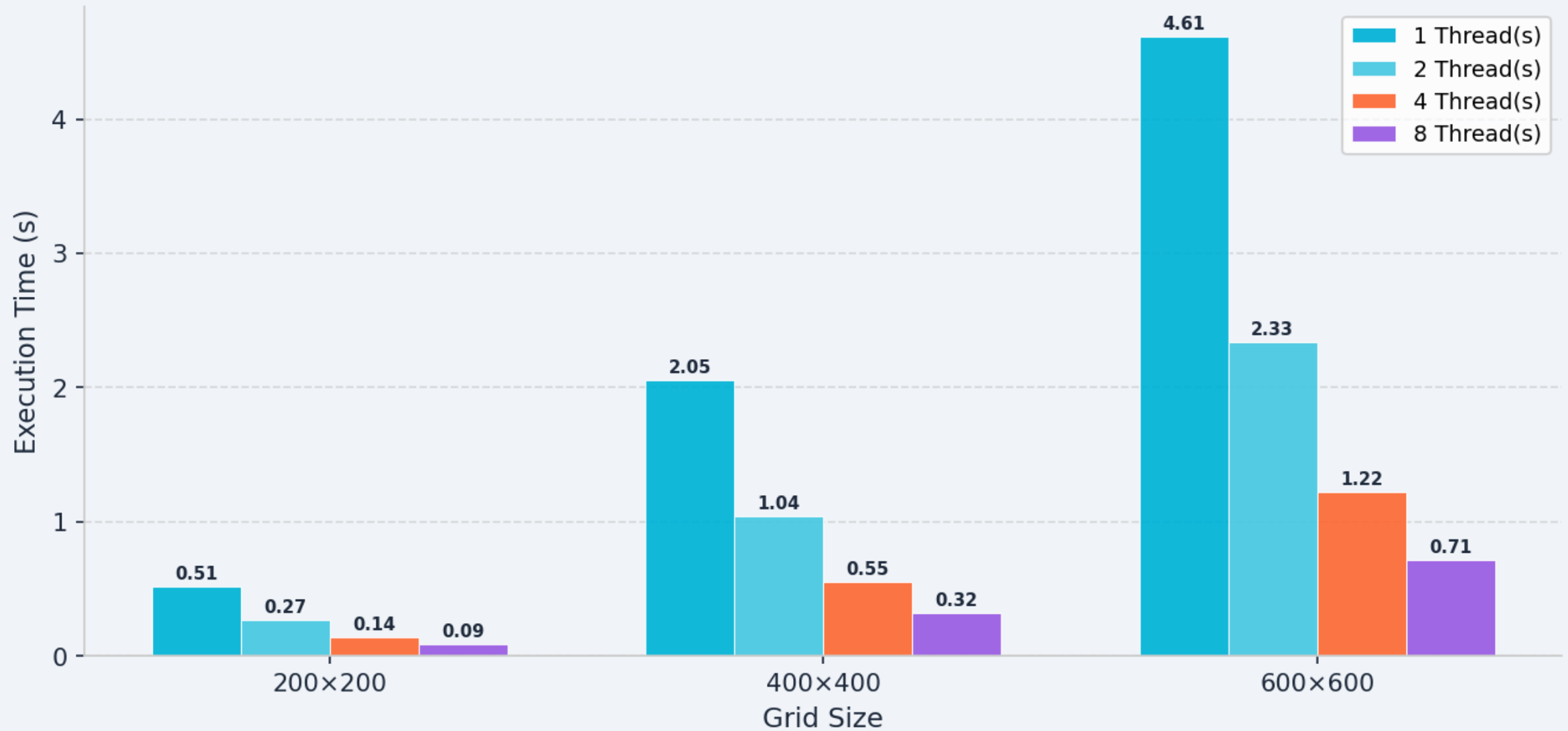
```

$ ./test
Grid Size | Threads | Parallel | Time (s) | Speedup
200x200 | 1 | Yes | 0.4863 | 0.9762
200x200 | 2 | Yes | 0.2892 | 1.6348
200x200 | 4 | Yes | 0.1806 | 2.9434
200x200 | 8 | Yes | 0.1962 | 2.4103
200x200 | 1 | No | 0.4728 | 1.0000
400x400 | 1 | Yes | 2.2106 | 0.9732
400x400 | 2 | Yes | 1.1751 | 1.8308
400x400 | 4 | Yes | 0.8007 | 2.5870
400x400 | 8 | Yes | 0.6795 | 3.1661

```

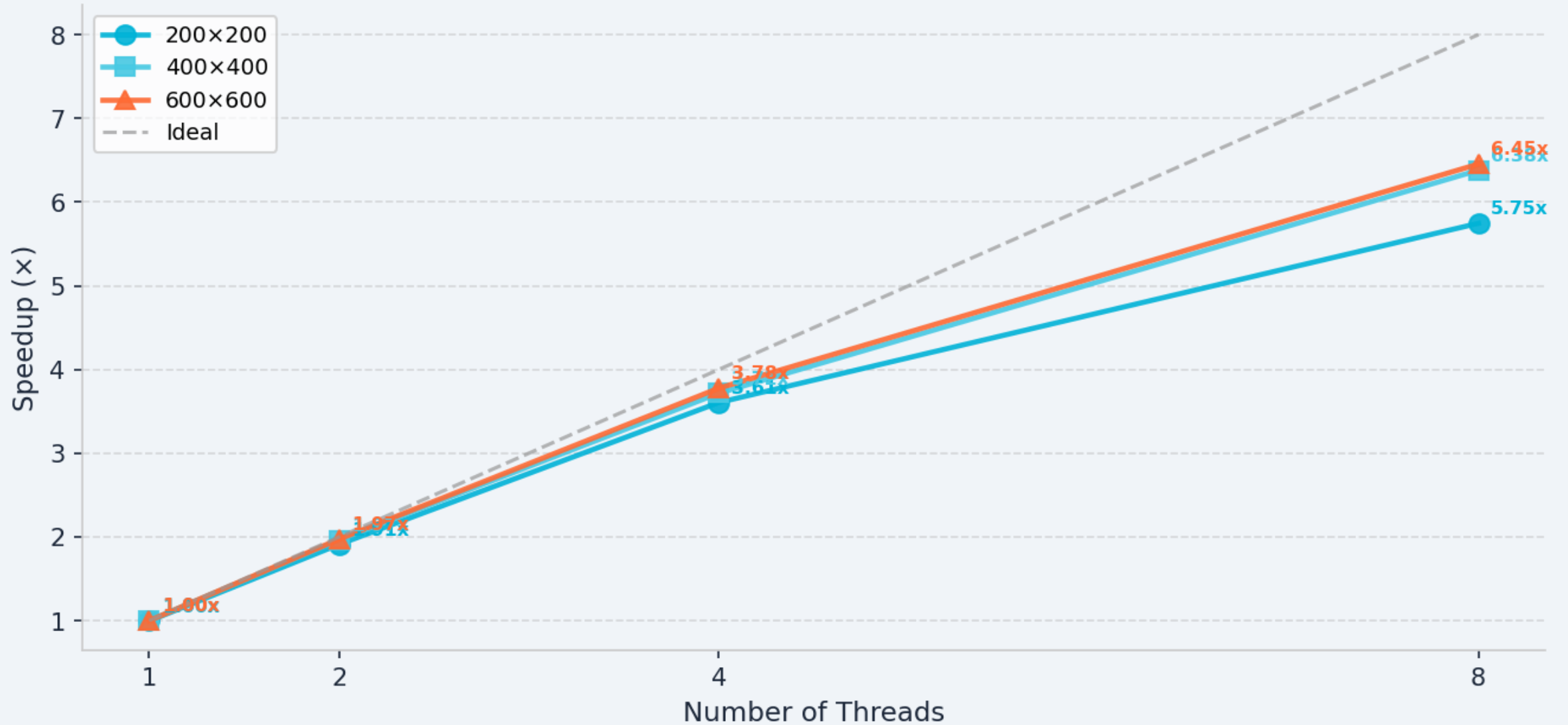
Execution Time — Parallel vs Threads

Lower is better • All values in seconds • 200 simulation steps



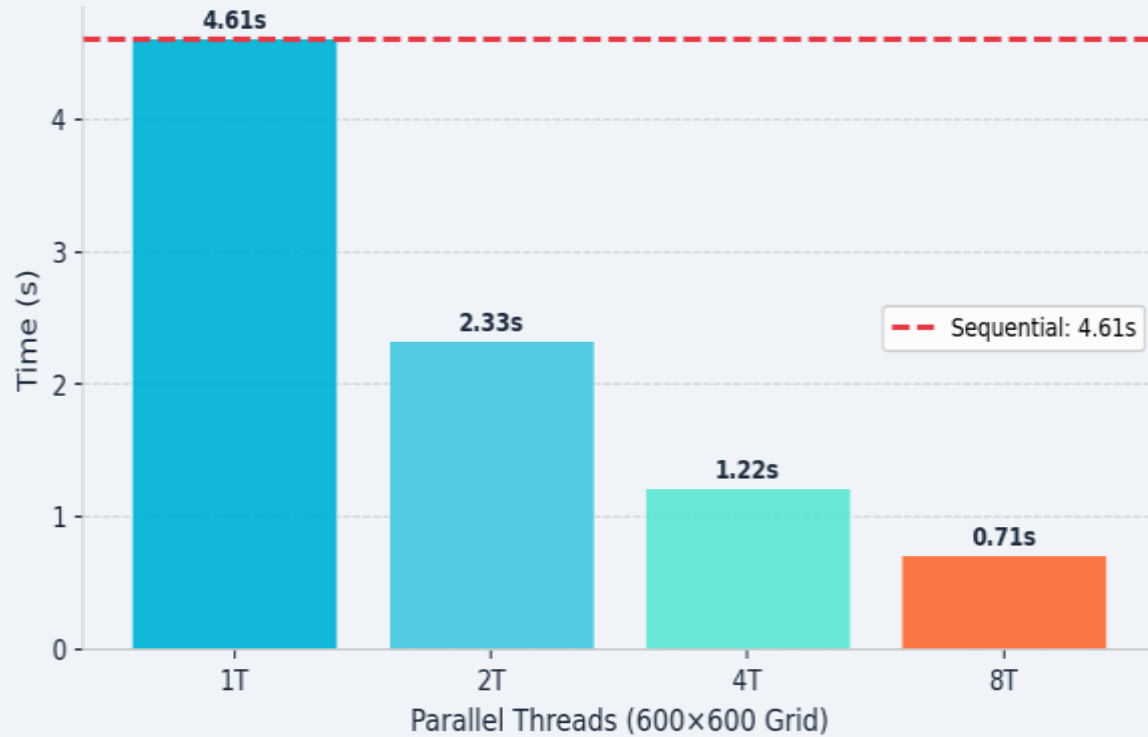
Speedup Analysis — All Grid Sizes

$\text{Speedup} = \text{Sequential Time} / \text{Parallel Time}$ • Dashed = ideal linear scaling

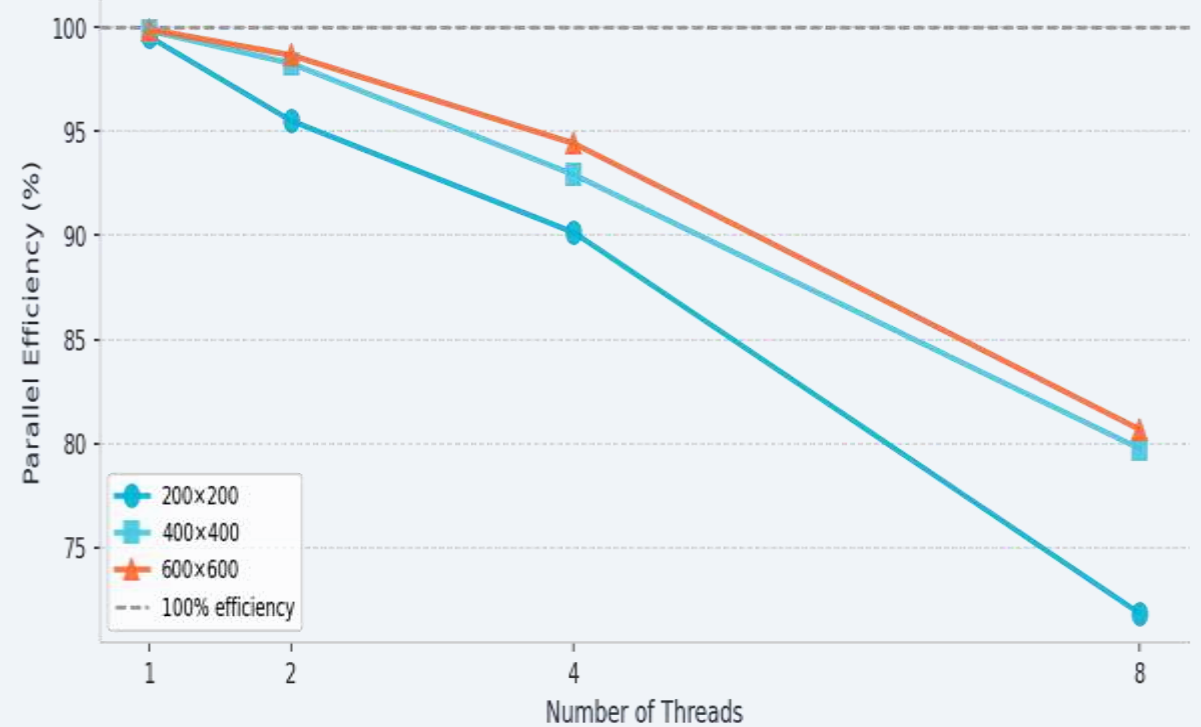


Sequential vs. Parallel & Parallel Efficiency

600×600 Grid: Seq vs Par



Parallel Efficiency by Grid Size



💡 Larger grids show better efficiency because computation-to-thread-overhead ratio is higher

Key Findings & Takeaways

Peak Speedup

~3.47×

600×600 grid with 8 threads — real hardware shows sub-linear but meaningful scaling

Scalability

Sub-linear

Efficiency drops from ~99% (2T) to ~80% (8T) as thread-management cost rises

collapse(2)

Correct

Flattening nested loops improves load balance across all threads evenly

Overhead Effect

~1.0×

1-thread parallel is almost identical to sequential — no false parallelism overhead

Grid Size Impact

Bigger = Better

Larger grids hide overhead — 600×600 outperforms 200×200 in efficiency at 8T

Static Schedule

Optimal

Static scheduling is ideal when all cells take equal compute time (which they do here)

Conclusion

This project successfully demonstrates how OpenMP shared-memory parallelism can accelerate grid-based environmental simulations. By parallelizing the inner cell-update loops with `collapse(2)` and static scheduling, we achieved up to 3.47× speedup on a 600×600 grid with 8 threads — a solid result for a CPU-only solution on real hardware.



OpenMP is highly effective for data-parallel, embarrassingly parallel grid workloads



Larger grid sizes amortize thread-creation overhead — scalability improves with problem size



Sequential baseline provides an honest comparison; 1-thread parallel $\approx$ sequential (no spurious gain)



The simulation is deterministic (`srand(42)`) and reproducible across all runs



Future work: GPU acceleration (CUDA/OpenCL), MPI for distributed-memory clusters, fire-probability parameter sweep